

تقرير شرح مشروع مساعد سرطان القولون والمستقيم

نسخة محسنة لاتجاه النص: العربي من اليمين، والمصطلحات الإنجليزية داخل عناصر مستقلة حتى لا تتقلب الجمل المختلطة.

البند	القيمة الحالية
اسم المشروع	Colorectal Cancer Assistant
نوع المشروع	مساعد أسئلة وأجوبة مبني على RAG
المصادر	NICE NG151 و جزء سرطان القولون والمستقيم من NICE NG12
عدد ملفات PDF	2
عدد chunks	95
موديل Embeddings	intfloat/multilingual-e5-base
قاعدة المتجهات	Chroma
القيمة الثابتة في التشغيل	Top-k = 5
أسئلة التقييم	130 total / 122 in-scope / 8 out-of-scope

1. فكرة المشروع باختصار

المشروع عبارة عن مساعد طبي متخصص في سرطان القولون والمستقيم. المستخدم يسأل بالعربية أو الإنجليزية، والسيستم يبحث في إرشادات رسمية من NICE ثم يرد بإجابة مختصرة ومعها مصدر واضح.

الفكرة الأساسية: الموديل لا يعتمد على ذاكرته العامة. الإجابة لازم تكون مبنية على النصوص المسترجعة من الدليل، ولو الدليل لا يغطي السؤال فالسيستم يرفض بأمان.

خط سير المشروع العام

NICE PDFs -> Parsing -> Cleaning -> Chunking -> Embeddings -> Chroma
Question -> Retrieval Top-5 -> LLM Generation -> Cited answer
Admin -> Analysis Page -> Evaluation metrics

تقسيم الشغل حسب أيام الهاكاثون

- اليوم الأول: اختيار مصدر رسمي، تنزيل PDF ، عمل parsing وتحويل النص إلى صيغة منظمة.
- اليوم الثاني: عمل chunks ، حساب embeddings ، التخزين في Chroma ، وبناء التقييم.
- اليوم الثالث: إضافة موديل توليد يكتب الإجابة من النصوص المسترجعة فقط.
- اليوم الرابع: بناء واجهة ويب وصفحة تحليل خاصة لعرض النتائج.

2. صفحة Analysis بتعرض إيه؟

صفحة التحليل موجودة في `web/components/dashboard.tsx` ، وهي صفحة خاصة محمية بكلمة سر. عند فتحها يتم تشغيل `evaluate.py --top-k 5 --json` من خلال `/api/analysis/evaluate` ثم عرض النتائج.

العنصر	المعنى	مكانه في الكود
Found Rate@5	هل الدليل المتوقع ظهر داخل أول خمس نتائج.	<code>evaluate.py -> build_summary -> found_rate</code>
Mean Precision@5	متوسط نسبة النتائج الصحيحة داخل الخمس نتائج.	<code>precision_at_k</code> ثم <code>mean_precision_at_k</code>
MAP@5	يقيس جودة ترتيب الأدلة الصحيحة داخل النتائج.	<code>average_precision_at_k</code> ثم <code>map_at_k</code>
MRR	يقيس ترتيب أول نتيجة صحيحة. كلما اقترب من 1 كان أفضل.	<code>reciprocal_rank</code> ثم <code>mrr</code>
Arabic / English	يقيس أداء الأسئلة العربية والإنجليزية منفصلين.	<code>ar_found_count</code> و <code>en_found_count</code>
Rank 1 / Top 3 / Top 5	يوضح أين ظهر أول دليل صحيح.	<code>rank_distribution</code>
Temporary Top-k	تجربة قيمة مختلفة مؤقتًا دون تغيير قيمة الإنتاج.	<code>runEvaluation</code> في <code>dashboard.tsx</code>

3. النتائج الحالية

المقياس	القيمة	التفسير
Found Rate@5	100.00%	122/122 سؤال داخل النطاق وجدنا لهم الدليل المتوقع.
Mean Precision@5	25.74%	طبيعي تكون أقل لأننا نرجع خمس قطع والسؤال غالبًا له قطعة صحيحة واحدة أو اثنتان فقط.
MAP@5	82.63%	رقم جيد لأنه يوضح أن الدليل الصحيح يظهر بترتيب متقدم غالبًا.
MRR	86.15%	أول دليل صحيح قريب جدًا من المركز الأول في أغلب الأسئلة.
Arabic	61/61	كل الأسئلة العربية داخل النطاق نجحت.
English	61/61	كل الأسئلة الإنجليزية داخل النطاق نجحت.

مقارنة Top-k

القيمة	Found Rate	Precision	الاستنتاج
k=1	77.05%	77.05%	دقيق لكنه يفوت بعض الأدلة.
k=3	95.90%	39.62%	توازن جيد.
k=4	97.54%	30.94%	تغطية أعلى من k=3.
k=5	100.00%	25.74%	الاختيار النهائي لأنه يغطي كل الأدلة المتوقعة.

4. نوع Chunking المستخدم

النوع المستخدم هو `Structure-aware Recursive Chunking` مع حساب الحجم بالـ `tokens`. يعني هو ليس تقسيم ثابت أعمى فقط، وليس `Semantic chunking`.

- `Document-aware`: لأن التقسيم يراعي أرقام التوصيات، الصفحات، العناوين، والجداول.
- `Recursive`: لأن `LangChain RecursiveCharacterTextSplitter` يجرب فواصل متعددة: فقرة، سطر، جملة، كلمة.
- `Token-aware`: لأن الحجم محسوب بتوكنيزر موديل `multilingual-e5-base`.
- حجم القطعة: `450 tokens`.
- التداخل: `80 tokens` لتقليل فقدان السياق بين القطع.

لو حد سألكم عن النوع: قولوا استخدمنا `Structure-aware recursive chunking with token overlap`.

5. المكتبات ولماذا استخدمناها

المكتبة	استخدامها	سبب الاختيار
pymupdf411m	تحويل PDF إلى Markdown وصفحات.	أفضل من استخراج نص خام لأنه يحافظ على بنية أوضح.
langchain-text-splitters	عمل Recursive chunking .	مكتبة مشهورة في مشاريع RAG وسهلة الشرح.
sentence-transformers	تحميل موديل intfloat/multilingual-e5-base وعمل embeddings .	مجاني ومفتوح المصدر ويدعم عربي وإنجليزي.
chromadb	تخزين المتجهات والبحث بالنتشابه.	محلي وبسيط ومناسب للديمو.
langchain-groq	تشغيل موديل التوليد السحابي.	أبسط من تشغيل موديل محلي، وأقل مشاكل من حدود Gemini في الديمو.
python-dotenv	قراءة مفاتيح API من ملف البيئة.	يمنع وضع المفاتيح داخل الكود.
Next.js + TypeScript	بناء الشات وصفحة التحليل وال-API routes.	واجهة حديثة مع types واضحة.
Tailwind CSS	تنسيق الواجهة بسرعة.	مناسب لتصميم Dark UI متناسق.
Recharts	رسوم بيانية في صفحة التحليل.	سهل ومناسب لل-dashboards.

6. الموديلات المستخدمة

الموديل	مكان الاستخدام	الدور
intfloat/multilingual-e5-base	ingest.py , query.py , evaluate.py	تحويل النص والسؤال إلى vectors للبحث الدلالي.
qwen/qwen3.6-27b	generate.py	صياغة إجابة قصيرة من الأدلة المسترجعة فقط.

مهم جدًا: موديل التوليد ليس مصدر المعرفة. مصدر المعرفة هو النصوص المخزنة من **NICE** ، والموديل يستخدمها فقط لصياغة الإجابة.

7. شرح الملفات الأساسية

config.py

ملف الإعدادات المركزي: المسارات، أسماء الموديلات، حجم القطع، **Top-k**، وروابط المصادر.

ingest.py

يبنى قاعدة المعرفة: قراءة **PDF**، تنظيف النص، تقسيمه، إضافة مصدر **NG12**، عمل **embeddings**، ثم التخزين في **Chroma**.

supplementary_sources.py

يستخرج توصيات سرطان القولون والمستقيم فقط من **NICE NG12**، بدون إدخال باقي أنواع السرطان.

query_understanding.py

يفهم صياغات السؤال قبل البحث: يطبع السؤال في سياق سرطان القولون والمستقيم، يحدد **intent**، وينظف العربي.

8. شرح باقي الملفات الأساسية

query.py

مسؤول عن الاسترجاع: يحمل موديل embeddings، يفتح Chroma، يوسع السؤال عند الحاجة، يبحث في قاعدة المتجهات، ويرتب النتائج.

generate.py

مسؤول عن الإجابة: يأخذ النتائج من query.py، يرسلها إلى موديل التوليد، يفرض شكل الإجابة، يتحقق من المصدر، ويرفض لو لا يوجد دليل كافٍ.

evaluate.py

مسؤول عن القياس: يقرأ test_questions.csv، يبحث عن الأدلة، يحسب Found Rate، Precision، MAP، MRR، ويحفظ النتائج.

web_chat_bridge.py

جسر بين Next.js و Python. يبقي موديل embeddings محملاً حتى لا يبدأ من الصفر مع كل سؤال.

9. ملفات الواجهة

الملف	وظيفته
web/app/page.tsx	الصفحة الرئيسية وتعرض الشات.
web/components/assistant-app.tsx	الهيدر، اسم الموقع، زر تغيير اللغة، وحالة جاهزية المساعد.
web/components/chat-panel.tsx	واجهة الشات، إرسال السؤال، loading، وعرض الإجابة مع زر NICE source.
web/app/api/chat/route.ts	يشغل Python bridge ويرسل له السؤال.
web/app/analysis/page.tsx	صفحة التحليل الخاصة بعد تسجيل الدخول.
web/components/dashboard.tsx	كروت metrics، الرسوم البيانية، تجربة k، وجدول الأسئلة.
web/lib/analysis-auth.ts	حماية صفحة التحليل بكلمة سر وcookie.
web/lib/data.ts	أرقام fallback للواجهة.

10. أهم الدوال في `evaluate.py`

وظيفةها	الدالة
قراءة أسئلة التقييم من CSV.	<code>load_questions</code>
استخراج أرقام التوصيات والتعامل مع <code>.ranges</code> .	<code>expected_recommendations</code>
تحديد هل <code>chunk</code> مطابق للدليل المتوقع مع التفرقة بين NG12 و NG151.	<code>is_relevant</code>
حساب <code>AP@k</code> .	<code>average_precision_at_k</code>
تشغيل البحث على كل الأسئلة وحساب نتيجة كل سؤال.	<code>evaluate</code>
حساب ملخص النتائج المعروض في صفحة <code>Analysis</code> .	<code>build_summary</code>
حفظ CSV و JSON للنتائج.	<code>save_reports</code>

11. أهم الدوال في query.py و generate.py

الدالة	وظيفتها
expand_question	إضافة صياغة قريبة من كلام NICE لتحسين البحث.
get_embedding_model	تحميل موديل embeddings مرة واحدة.
get_collection	فتح Chroma collection مرة واحدة.
search	البحث واسترجاع أفضل chunks.
SYSTEM_PROMPT	قواعد تمنع الهلوسة وتلزم الموديل بالمصدر.
citation_for	بناء citation من metadata.
is_valid_answer	التحقق من شكل الإجابة والمصدر.
claims_are_supported	فحص إضافي أن الادعاءات مدعومة بالأدلة.
generate	الدالة الرئيسية التي تجمع retrieval + generation + refusal.

12. نقاط مهمة في المناقشة

- السيستم متخصص في سرطان القولون والمستقيم فقط، وليس chatbot طبي عام.
- لو السؤال خارج الداتا، الأفضل رفض الإجابة بدل اختراع إجابة.
- `Precision@5` الأقل لا يعني فشل؛ لأننا نرجع 5 نتائج بينما الدليل المتوقع غالبًا قطعة واحدة.
- `Found Rate@5` هو الأهم لأنه يؤكد وصول الدليل الصحيح إلى موديل التوليد.
- استخدمنا `multilingual-e5-base` لأنه مفتوح المصدر ويدعم العربي والإنجليزي.
- اختيار `Chroma` لأنه سريع وبسيط ومناسب لنسخة الهاكاثون.
- أول سؤال قد يأخذ وقتًا بسبب تحميل الموديل، وبعدها `web_chat_bridge.py` يقلل الوقت.